

FoodChow AI Support Agent – Interview Assignment

Build an Agentic Customer Support System for FoodChow that can understand customer issues, retrieve relevant knowledge, investigate problems using tools, take controlled actions, and escalate complex cases to human support.

1. Assignment Objective

The candidate should build an AI-powered Customer Support Agent for FoodChow.

This should not be a simple FAQ chatbot. The system should behave like an Agentic AI Support System capable of:

- Understanding customer problems
- Searching a knowledge base
- Asking relevant follow-up questions
- Calling diagnostic tools/APIs
- Identifying possible root causes
- Suggesting or performing allowed actions
- Maintaining conversation context
- Escalating unresolved or sensitive issues to human support

2. Example Support Categories

The system may handle the following categories:

POS

- POS not syncing orders
- Incorrect GST or billing
- POS offline
- Printer not printing

KDS

- Orders not appearing on the kitchen screen
- Kitchen not receiving online orders

Online Ordering

- Customers cannot place orders
- Menu changes are not appearing
- Items showing available when sold out

Payments

- Payment deducted but order remains pending
- Payment successful but order failed

Restaurant Setup

- Add a new outlet
- Add or update a menu
- Configure a printer

Account

- Password reset
- Update restaurant details

3. Expected Agent Workflow

Customer Message



Understand and classify the issue



Search relevant Knowledge Base



Determine whether additional information is required



Ask follow-up questions if needed



Call diagnostic tools/APIs



Analyze results and identify likely root cause



Provide resolution or take an allowed action



Ask whether the issue is resolved



Resolved → Close conversation

Not resolved / sensitive → Escalate to human support

4. Knowledge Base / RAG Requirement

The candidate should implement a Knowledge Base or RAG system containing documentation related to:

- POS
- KDS
- Printers

- Online Ordering
- Payments
- Menu Management
- Restaurant Setup
- Accounts
- Troubleshooting

The AI should retrieve relevant information instead of relying entirely on a large static prompt.

5. Tool Calling Requirement

Provide mock APIs or tools that the AI Agent can call. Suggested tools:

- `get_restaurant()`
- `get_outlet()`
- `get_order()`
- `get_order_status()`
- `get_payment_status()`
- `get_printer_status()`
- `get_kds_status()`
- `get_menu_status()`
- `create_support_ticket()`

The candidate may create mock APIs or simulated backend responses.

6. Example Agentic Scenario

Customer:

“My printer isn't working.”

Expected behavior:

1. Identify the issue as a printer/hardware problem.
2. Search the knowledge base for relevant troubleshooting information.
3. Ask for additional information if required, such as outlet or printer.
4. Call `get_printer_status()`.
5. Check the result.
6. Optionally check `get_last_successful_print()`.
7. Explain the likely issue.
8. Provide troubleshooting steps.
9. If unresolved, create or escalate a support ticket.

The agent should investigate the problem rather than simply provide a generic FAQ answer.

7. Conversation Memory

The agent should maintain conversation context.

Example:

Customer: "Order #1024 is stuck."

Agent investigates the order.

Customer: "What about my previous order?"

The agent should understand the context and ask for clarification or correctly identify the referenced conversation context when possible.

8. Human Handoff and Escalation

The AI should not attempt to solve every problem.

Suggested escalation cases:

- Low confidence responses
- Payment disputes
- Refunds requiring approval
- Account security issues
- Critical POS outages
- Repeated unsuccessful troubleshooting
- Backend/API failures

When escalating, the system should transfer useful context, including:

- Customer issue
- Steps already performed
- Tool/API results
- Relevant order or restaurant information
- Conversation summary

9. Permissions and Controlled Actions

Sensitive actions should require appropriate controls.

Example:

Customer: "Refund my order."

Expected workflow:

AI

↓

Check order

↓
Check payment status
↓
Check refund eligibility
↓
Calculate refund amount
↓
Determine whether approval is required
↓
Request approval or escalate
↓
Execute only if authorized

The AI must not casually execute destructive or sensitive actions such as refunds, account deletion, or major configuration changes.

10. Failure Handling

The system should gracefully handle failures such as:

- API timeout
- Invalid tool response
- Missing order
- Unknown restaurant
- API unavailable
- Invalid JSON
- Conflicting information

The agent should not hallucinate information when a tool fails.

11. Evaluation Scenarios

The candidate should demonstrate the system using predefined scenarios.

Example:

Customer:

“Payment was deducted but my order isn't confirmed.”

Expected reasoning flow:

1. Understand the payment/order issue.
2. Retrieve relevant knowledge.
3. Get order information.
4. Get payment information.
5. Compare order and payment states.

6. Identify the likely issue.
7. Explain the result clearly.
8. Take an allowed action or escalate if necessary.

Evaluation should consider both the final answer and how effectively the agent uses knowledge and tools.

12. Technical Expectations

The implementation should ideally demonstrate:

- LLM integration
- RAG / Knowledge Retrieval
- Tool or Function Calling
- Conversation Memory
- Agent Workflow / Orchestration
- Error Handling
- Guardrails
- Human Handoff
- Logging or Observability

The candidate may choose the technology stack unless specific requirements are provided.

13. Submission Requirements

The candidate should submit:

- Source code
- README with setup instructions
- Architecture overview
- List of tools/APIs implemented
- Sample knowledge base
- Demonstration scenarios
- Explanation of agent workflow
- Known limitations and future improvements

14. Bonus Points

Bonus points may be awarded for:

- Multi-agent architecture
- Evaluation framework
- Agent tracing/observability
- Role-based permissions
- Confidence scoring

- Automatic retry strategies
- Multilingual support
- Web-based support dashboard
- Human agent handoff interface
- Cost and token optimization

15. Difficulty Level

Recommended target for this interview assignment:

RAG + Tool Calling + Conversation Memory + Controlled Actions + Human Handoff

This level provides a good balance between evaluating real Agentic AI skills and keeping the assignment practical.

Final Assignment Statement

Build an AI Support Agent for FoodChow that can diagnose and resolve customer issues using knowledge retrieval, tool calling, conversation context, controlled actions, and human escalation. The objective is to evaluate how effectively the candidate designs an Agentic AI system rather than a simple FAQ chatbot.